PES UNIVERSITY
Department of Computer Science and Engineering
Software Engineering | UE23CS341A

SECURITY LOG

Vehicle Parking System

Security Design and Architecture (Unit 2, Lectures 2-3)

Project repository	github.com/Sam-0960/Vehicle-Parking-System---SE---D-sec
Prepared for	Software Engineering group project
Purpose	To list possible security problems, the checks we should do, and what needs to be fixed or confirmed.
Based on	Project documents and the course notes on Security Design and Architecture.
Document version	v0.1 (Draft)

1. Introduction and Security Objectives

1.1 Purpose and scope

This Security Log records the security posture of the Vehicle Parking System (VPS), a Software Engineering course project. It applies the Secure Architecture Design approach from Unit 2 (Security Design and Architecture): embed security principles into architectural decisions, reduce the attack surface, keep concerns separated, support defence-in-depth, and align controls with threat models and security requirements. The scope covers the application components, data flows, deployment environment and integration points (for example, the payment gateway).

The log is a living record. It contains the risk register, a security event log, the controls (with honest implementation status), a test checklist and an incident response plan.

> **Drafting note (delete before submission).** This draft was prepared from the lecture notes. The project repository could not be opened automatically while drafting, so component names, roles and flows below are typical for a parking system and are tagged as assumptions. Please verify them against your SRS and design documents and edit before submitting. No incidents or test results have been invented: every control is marked Proposed and every test is marked Not executed until your team records evidence.

1.2 Security objectives

Following the lecture example (patient data must be confidentiality, integrity and replay protected), the objectives for VPS are:

- **Confidentiality:** user profiles, vehicle numbers, reservation history and payment details are visible only to authorised parties.
- **Integrity:** slot availability, reservations, tariffs and payment records cannot be altered except through authorised workflows.
- **Replay protection:** captured reservation or payment requests cannot be resubmitted to create duplicate or fraudulent transactions.
- **Availability:** slot booking and entry/exit processing remain usable under abusive or heavy traffic.
- **Accountability:** security-relevant actions (login, role change, refund, slot override) are logged and traceable to a user.

1.3 Status terminology used in this log

Status	Meaning
Proposed	Control is designed or recommended but not yet built in the project.
Implemented	Control exists in the code or configuration. Must cite evidence (commit, PR or screenshot).
Verified	Implemented and confirmed by a security test with recorded evidence.

2. Important Assets and Potential Threats

2.1 Asset identification across the data lifecycle

The lecture asks us to consider assets at their origin point, during transit, at rest and in use. The table applies this to the main VPS assets (component names marked [A] are assumptions to confirm).

Asset	Origin	In transit	At rest	In use
User credentials and profile (name, phone, email)	Registration / login page [A]	Client to web server	User table in database	Profile view, login check
Vehicle and reservation data (plate number, slot, time)	Booking form, entry gate or attendant desk [A]	Client to application server; app to DB	Reservation table	Slot display, attendant verification

Asset	Origin	In transit	At rest	In use
Payment and billing records	Payment page / exit counter [A]	App to payment gateway [A]	Billing table (no card data planned)	Receipts, admin reports
Slot inventory and tariff configuration	Admin dashboard [A]	Admin client to server	Slot and tariff tables	Availability search, fee calculation
Admin accounts and audit logs	Admin setup, system events	App to log store	Log files / table	Security review

Table 1: Assets across the lifecycle.

2.2 Trust boundaries and adversaries

Mirroring the lecture (legitimate users and staff inside the boundary; others outside), the proposed trust model is:

- **Inside the trust boundary:** authenticated customers (for their own data only), parking attendants and administrators acting within their role.
- **Outside the trust boundary:** anonymous internet users, other customers (for each other's data), unauthenticated API callers, and the payment gateway (trusted only after signature verification).
- **Protection required from:** other customers, external attackers, and insider threat (a malicious or careless staff member or administrator).

Boundaries to be drawn on the architecture diagram: Client/Browser to Application Server; Application Server to Database; Application Server to Payment Gateway; Admin interface to Application Server.

2.3 Potential threats

Threat category	Description in the VPS context
Unauthorized access	Guessing or stealing credentials, weak password rules, no lockout, access to pages without login.
SQL injection	Malicious input in login, search or vehicle-number fields altering database queries.
Privilege escalation	A customer reaching admin or attendant functions by forced browsing or tampering with role parameters.
Data leakage	Exposure of personal and vehicle data via verbose errors, logs, unencrypted traffic or over-broad API responses.
Reservation manipulation	Changing another user's booking (IDOR), double-booking a slot through race conditions, altering time, slot or price fields.
Payment security	Tampering with the amount, replaying a payment request, forging gateway callbacks, or storing card data.
Session attacks and replay	Stolen or non-expiring session tokens; replay of captured requests.
Denial of service / abuse	Request floods or bots hoarding slots by creating many unpaid reservations.

3. Risk Assessment

Likelihood (L) and Impact (I) are rated Low = 1, Medium = 2, High = 3. Risk level is the product: 1-2 Low, 3-4 Medium, 6 High, 9 Critical. Ratings are initial qualitative estimates for team review, not measured values.

ID	Threat	Affected component	L	I	Risk level	Mitigation (control ref.)
R-01	Unauthorized access via weak or stolen credentials, no lockout	Authentication module, login API	Medium	High	High	Strong password policy, hashed storage (bcrypt/Argon2), lockout and throttling, MFA for admins (C-01, C-07)
R-02	SQL injection through unvalidated input	Database access layer, search and login forms	Medium	High	High	Parameterised queries/ORM, input validation layer, least-privilege DB account (C-03, C-10)
R-03	Privilege escalation (customer to attendant/admin)	Authorisation layer, admin dashboard	Medium	High	High	Server-side RBAC, deny by default, role never taken from client input (C-02)
R-04	Data leakage of personal and vehicle data	API responses, logs, DB, network	Medium	High	High	TLS everywhere, encryption at rest for sensitive fields, generic errors, minimal API output, log hygiene (C-04, C-09)

ID	Threat	Affected component	L	I	Risk level	Mitigation (control ref.)
R-05	Reservation manipulation (IDOR, double booking, parameter tampering)	Reservation module, slot inventory	High	Medium	High	Ownership checks, transactional slot locking, server-side price/time calculation, non-guessable IDs (C-05)
R-06	Payment tampering, replay or forged callback	Payment module, gateway integration	Medium	High	High	Hosted gateway/tokenisation, no card storage, idempotency keys, signed callbacks, server-side amount (C-06, C-04)
R-07	Session hijacking and request replay	Session management	Medium	Medium	Medium	Secure/HttpOnly cookies, short expiry, rotation on login, nonce or idempotency on sensitive calls (C-01, C-04)
R-08	API abuse and denial of service (slot hoarding)	Public APIs, booking endpoint	Medium	Medium	Medium	Rate limiting, reservation hold timeout, CAPTCHA on repeated attempts (C-07)
R-09	Insecure defaults and misconfiguration	Deployment, admin account, framework settings	Low	High	Medium	Secure configuration baseline, no default credentials, debug off, secrets outside code (C-08)
R-10	Insider misuse (malicious staff or admin)	Admin dashboard, DB, logs	Low	High	Medium	Least privilege, separation of duties, audit logging of admin actions (C-02, C-09, C-10)
R-11	Insufficient logging and monitoring	Logging subsystem	Medium	Medium	Medium	Centralised audit logs for auth, payment and admin events; alerts on anomalies (C-09)

Table 2: Risk register. Components are assumptions [A] pending confirmation against the project design.

4. Security Event Log

This table records real security-relevant events only. Entries SEC-001 and SEC-002 describe documentation activity. The remaining rows are blank templates: the team should add an entry whenever a finding, test failure, vulnerable dependency alert, suspicious activity or configuration issue is observed, and attach evidence. Severity scale: Informational, Low, Medium, High, Critical.

Log ID	Date	Event	Severity	Action	Status	Evidence
SEC-001	14-Oct-2026	Security Log created; initial risk register drafted from the Security Design and Architecture lecture checkpoints	Informational	Draft circulated to team for review	Closed	This document (v0.1)
SEC-002	[Date]	Cross-check of risk register and component names against project SRS/design documents in repository	Informational	Team to confirm assumptions [A] and update Tables 1-3	Open	[Commit or document reference]
SEC-003	[YYYY-MM-DD]	[Event description]	[Severity]	[Action taken]	[Open / Closed]	[Screenshot, log excerpt, issue link]
SEC-004	[YYYY-MM-DD]	[Event description]	[Severity]	[Action taken]	[Open / Closed]	[Screenshot, log excerpt, issue link]
SEC-005	[YYYY-MM-DD]	[Event description]	[Severity]	[Action taken]	[Open / Closed]	[Screenshot, log excerpt, issue link]

Table 3: Security event log. No security incidents are recorded because none have been reported to this log.

5. Security Controls and Mitigation Strategies

Controls follow the lecture techniques: input validation layers, secure communication channels (TLS, mTLS), encapsulation and service boundaries, and trust boundaries between subsystems. **All controls are currently Proposed.** Change a status to Implemented only when evidence is added, and to Verified only after the matching test in Section 6 passes.

ID	Control / mitigation strategy	Risks	Lecture technique	Status	Evidence
C-01	Authentication hardening: password policy, salted hash (bcrypt/Argon2), lockout, session expiry and rotation, MFA for admin	R-01, R-07	Secure design patterns	Proposed	[Evidence]
C-02	Role-based access control with deny-by-default; every request authorised server-side; separation of customer, attendant and admin roles	R-03, R-05, R-10	Trust boundary, access policy	Proposed	[Evidence]
C-03	Input validation layer and parameterised queries (no string-built SQL); allow-list validation of plate numbers, dates and amounts	R-02	Input validation layers	Proposed	[Evidence]
C-04	TLS for all client and gateway traffic; HTTP redirected to HTTPS; encryption at rest for sensitive fields; mTLS if services are split	R-04, R-06, R-07	Secure communication channels	Proposed	[Evidence]
C-05	Reservation integrity: ownership checks, unpredictable IDs, transactional slot locking, server-side fee and time calculation	R-05	Encapsulation, validation	Proposed	[Evidence]
C-06	Payment protection: hosted payment gateway or tokenisation, no card number or CVV stored, idempotency keys, verified callbacks	R-06	Asset protection in transit and at rest	Proposed	[Evidence]
C-07	Rate limiting on login, booking and payment APIs; reservation hold timeout	R-01, R-08	APIs authenticated, rate-limited, validated	Proposed	[Evidence]
C-08	Secure configuration baseline: no default credentials, debug disabled, secrets in environment variables, minimal exposed ports	R-09	Secure defaults	Proposed	[Evidence]
C-09	Audit logging of login, role change, refund and slot override; logs exclude passwords and payment data; access to logs restricted	R-04, R-10, R-11	Defence-in-depth, review report	Proposed	[Evidence]
C-10	Subsystem separation: database not reachable from clients; application uses a least-privilege DB account; admin interface isolated	R-02, R-04, R-10	Trust boundaries, isolation	Proposed	[Evidence]

Table 4: Security controls and implementation status.

5.1 Architecture review checkpoints (from lecture)

Checkpoint	Current answer	Notes / evidence
Are all sensitive components isolated?	[Yes / Partial / No]	[Describe isolation of DB, payment, admin]
Are APIs authenticated, rate-limited and validated?	[Yes / Partial / No]	[List endpoints reviewed]
Is encryption used where needed (in transit and at rest)?	[Yes / Partial / No]	[TLS config, field encryption]
Are default configurations secure?	[Yes / Partial / No]	[Config review reference]

Architecture-phase outputs expected by the lecture: security-annotated architecture diagrams (with trust boundaries), a Security Architecture Review Report, change requests for gaps, and an Architecture Threat Map linking each high-value component to its threats. Table 2 provides the starting point for the threat map.

6. Security Testing Checklist

Expected results are what a correctly secured system should do. The Result column starts as Not executed. Record Pass or Fail, the date and evidence only after the test is actually run.

ID	Test	Risk	Expected result	Result	Evidence
T-01	Enter wrong password repeatedly (e.g. 5+ times)	R-01	Generic error, throttle or lockout applied, attempt logged	Not executed	[Evidence]
T-02	Open protected page or API without logging in	R-01	401 or redirect to login; no data returned	Not executed	[Evidence]

ID	Test	Risk	Expected result	Result	Evidence
T-03	Submit SQL metacharacters (e.g. a quote and OR 1=1) in login, search and plate fields	R-02	Input rejected or treated as literal text; no SQL error shown; no data change	Not executed	[Evidence]
T-04	Customer account calls admin or attendant URL/API directly	R-03	403 Forbidden; attempt logged	Not executed	[Evidence]
T-05	Change user or reservation ID in a request to view another user's booking	R-05, R-04	403 or 404; no foreign data returned	Not executed	[Evidence]
T-06	Modify amount or duration in the payment request	R-06	Server recomputes fee; tampered request rejected	Not executed	[Evidence]
T-07	Resend a captured booking or payment request	R-06, R-07	Duplicate rejected or idempotent; no second charge or booking	Not executed	[Evidence]
T-08	Two users book the same slot at the same time	R-05	Only one booking succeeds; the other receives a clear conflict message	Not executed	[Evidence]
T-09	Inspect traffic and try plain HTTP	R-04	Only TLS used; HTTP redirected to HTTPS	Not executed	[Evidence]
T-10	Send malformed input to trigger an error	R-04, R-09	Generic error page; no stack trace or SQL details	Not executed	[Evidence]
T-11	Inspect database contents for credentials and payment data	R-04, R-06	Passwords hashed; no card number or CVV stored	Not executed	[Evidence]
T-12	Log out, then reuse the old session token; check cookie flags	R-07	Token invalid after logout or expiry; cookies Secure and HttpOnly	Not executed	[Evidence]
T-13	Send a burst of requests to login and booking APIs	R-08	Rate limit applied (e.g. HTTP 429); service stays responsive	Not executed	[Evidence]
T-14	Review logs after tests T-01 to T-13	R-11, R-04	Auth, admin and payment events present; no passwords or payment data in logs	Not executed	[Evidence]
T-15	Check for default accounts and debug mode in deployment	R-09	No default credentials; debug disabled	Not executed	[Evidence]

Table 5: Security test checklist. Results are intentionally blank until tests are executed.

7. Incident Response and Conclusion

7.1 Incident severity and response

Severity	Example in VPS	Proposed response target
Critical	Confirmed data breach, payment fraud, admin account takeover	Immediate containment; notify faculty/stakeholders [placeholder]
High	Successful SQL injection or privilege escalation in testing or production	Contain and begin fix the same day
Medium	Repeated failed logins, rate-limit bypass, misconfiguration found	Fix within the current sprint
Low	Minor information disclosure, missing header	Schedule in backlog

Table 6: Incident severity classification (targets are proposals for the team to adopt or change).

7.2 Response process

1. **Detect and report:** identify the event from logs, test results or a user report; record it in the Security Event Log (Section 4) with date and evidence.
2. **Assess:** assign severity using Table 6 and identify the affected component and assets from Tables 1 and 2.
3. **Contain:** disable the affected account, endpoint or feature; revoke sessions or keys; preserve logs and screenshots.
4. **Eradicate and fix:** remove the root cause (patch code, correct configuration), then raise a change request for any design gap.
5. **Recover:** restore normal service, retest with the relevant checklist item, and update the control status.

6. **Review:** record lessons learned, update the risk register and threat map, and close the log entry.

7.3 Roles

Role	Responsibility	Assigned to
Incident lead	Coordinates response and decisions	[Name]
Technical owner	Investigates and applies the fix	[Name]
Documentation / log keeper	Maintains this log and evidence	[Name]
Faculty contact	Informed of High and Critical events	[Name]

7.4 Conclusion

This log establishes a structured security baseline for the Vehicle Parking System using the Secure Architecture Design principles taught in Unit 2: defined assets, trust boundaries, a risk register covering unauthorized access, SQL injection, privilege escalation, data leakage, reservation manipulation and payment security, and a mapped set of controls and tests. At the time of writing, all controls are proposed rather than implemented, no security tests have been run, and no incidents have been recorded.

Next steps for the team: confirm the assumptions against the project documentation, prioritise controls for the High-rated risks (R-01 to R-06), implement them with evidence, execute the checklist in Section 6, and keep Sections 3 to 6 updated as the project evolves.

Reference: Software Engineering, Unit 2 Lecture 2-3, Security Design and Architecture, Department of CSE, PES University (UE23CS341A).